OPENSHORTS — REFINED ARCHITECTURAL REPAIR & IMPLEMENTATION PLAN (v2)

Project: FluxClip Studio / OpenShorts Lightweight Frontend-Driven Pipeline | **Status:** Approved Architecture Refinements | **Version:** 2.0

1. Raw Binary Chunk Uploads (application/octet-stream) Instead of Base64 JSON

Problem with Base64 JSON

Encoding binary chunks into Base64 inside JSON inflates payload size by ~33%, increases memory allocation on both the browser/Android client and Node.js heap (`JSON.parse`), and blocks the UI thread during string conversion.

Refined Design

- **Endpoint:** `POST /api/upload/chunk`
- **Route-Scoped Middleware:** Mount `express.raw({ type: "application/octet-stream", limit: "8mb" })` specifically on `/api/upload/chunk` so `req.body` is received directly as a Node.js `Buffer` with zero Base64 or JSON parsing overhead.
- **Transport Headers:**
  - `Content-Type: application/octet-stream`
  - `X-Upload-Id: <uploadId>`
  - `X-Chunk-Index: <0-based integer>`
  - `X-Byte-Offset: <exact byte offset in file>`
  - `X-Chunk-Sha256: <optional hex digest or byte length check>`
- **Frontend & Android Client Implementation:**
  - **Web (`useOpenShortsViewModel.ts`):** Slice `File` directly with `const blobSlice = file.slice(byteOffset, byteOffset + chunkSizeBytes)` and pass `body: blobSlice` directly to `fetch("/api/upload/chunk", ...)` with the headers above. Zero `FileReader` or `window.btoa` conversion is needed.
  - **Android (`OpenShortsViewModel.kt`):** Read fixed-size `ByteArray` buffers from `ContentResolver.openInputStream(uri)` and stream raw `application/octet-stream` request bodies with `X-Upload-Id` , `X-Chunk-Index` , and `X-Byte-Offset` headers.

2. Idempotent Chunk Retries & Duplicate-Write Prevention

Problem with `fs.appendFileSync`

If a chunk request succeeds on the server but the response drops over a mobile network, the client retries the same `(chunkIndex, byteOffset)` . Using `appendFileSync` appends duplicate bytes and corrupts the media container.

Refined Design

Session State Tracking (`ChunkUploadSession`):

```
interface ChunkUploadSession {
  uploadId: string;
  kind: "video" | "audio";
  fileName: string;
  mimeType: string;
  totalSizeBytes: number;
```

```
  chunkSizeBytes: number;
  receivedChunks: Map<number, { byteOffset: number; byteLength: number }>;
  receivedBytes: number;
  tempFilePath: string;
  verified: boolean;
  probedDurationMs?: number;
  probedWidth?: number;
  probedHeight?: number;
  createdAt: number;
  lastActivityAt: number;
}
```

Positional File Writing (`pwrite` semantics):

1. Validate `byteOffset >= 0` , `buffer.byteLength > 0` , and `byteOffset + buffer.byteLength <= session.totalSizeBytes` .
2. **Idempotency Check:** If `session.receivedChunks.has(chunkIndex)` :
   - Verify that the recorded `{ byteOffset, byteLength }` matches the retried chunk.
   - Overwrite at the exact positional offset using `fs.openSync(session.tempFilePath, "r+")` + `fs.writeSync(fd, buffer, 0, buffer.byteLength, byteOffset)` (or return `200 OK` immediately without incrementing `session.receivedBytes`).
3. **First-Time Chunk Write:**
   - Enforce sequential offset continuity (`byteOffset === session.receivedBytes`) so sparse holes cannot occur.
   - Write buffer at exact offset `byteOffset` via `fs.writeSync(fd, buffer, 0, buffer.byteLength, byteOffset)` and close `fd` in a `finally` block.
   - Record `session.receivedChunks.set(chunkIndex, { byteOffset, byteLength: buffer.byteLength })` and recompute `session.receivedBytes` as the sum of unique recorded chunk lengths.
4. **Finalization (`POST /api/upload/complete`):** Verify `session.receivedBytes === session.totalSizeBytes` and `fs.statSync(session.tempFilePath).size === session.totalSizeBytes` before running `ffprobe` and marking `session.verified = true` .

3. Runtime Verification of `ffmpeg` & `ffprobe` Availability in the Deployment Environment

Problem

Binary paths can differ across local containers, Cloud Run revisions, or custom self-hosted environments (`/usr/bin/ffmpeg` , `/usr/local/bin/ffmpeg` , or `FFMPEG_PATH` / `FFPROBE_PATH` env overrides).

Refined Design

- **Startup & Per-Job Capability Probe (`resolveMediaBinaries()`):**
  1. Check `process.env.FFMPEG_PATH` / `process.env.FFPROBE_PATH` first, then standard system paths (`/usr/bin/ffmpeg` , `/usr/local/bin/ffmpeg` , `ffmpeg` on `PATH`).
  2. Execute `execFile(ffmpegBin, ["-version"], { timeout: 5000 })` and `execFile(ffprobeBin, ["-version"], { timeout: 5000 })` at server startup and cache the verified binary paths and version strings.
  3. Expose the verified binary status in `GET /api/me` (`mediaEngine: { ffmpegAvailable: boolean, ffprobeAvailable: boolean }`).

4. In `POST /api/upload/complete` and `POST /api/process`, if either binary is unavailable or fails verification, return HTTP `503 Service Unavailable` (`MEDIA_ENGINE_UNAVAILABLE`) with a clear diagnostic message instead of queueing a job that will fail mid-pipeline.

4. Aspect-Ratio-Adaptive Video Crop & Scale Logic (`ffprobe` Stream Dimensions)

Problem with Hardcoded `-vf crop=ih*9/16:ih,scale=1080:1920`

A hardcoded `crop=ih*9/16:ih` assumes the input video is always wider than `9:16` (such as 16:9 landscape). If the source video is already vertical (`9:16`) or narrower than 9:16, `ih*9/16` can exceed `iw`, causing FFmpeg to fail with an invalid crop size error.

Refined Design

- **Step A — Probe Source Video Geometry via `ffprobe`:**
  - Extract `width`, `height`, and rotation/display aspect ratio from the first video stream (`-select_streams v:0 -show_entries stream=width,height,side_data_list:format=duration -of json`).
  - Account for rotation metadata (90 or -90 degrees swaps effective width and height).
- **Step B — Adaptive 9:16 Filter Expression (`buildAdaptive9x16VideoFilter`):**
  Target output canvas: even-dimensioned `9:16` (720x1280 or 1080x1920 divisible by 2 for `yuv420p`):
  - **Case 1: Landscape or Square Input (`sourceAspect > 9/16`, e.g. 16:9, 4:3, 1:1):**
    `crop='trunc(ih*9/16/2)*2:trunc(ih/2)*2:(iw-trunc(ih*9/16/2)*2)/2:0',scale=720:1280,setsar=1`
  - **Case 2: Already 9:16 Input (`|sourceAspect - 9/16| < 0.01`):**
    `scale=720:1280,setsar=1`
  - **Case 3: Narrower than 9:16 Input (`sourceAspect < 9/16`):**
    `crop='trunc(iw/2)*2:trunc(iw*16/9/2)*2:0:(ih-trunc(iw*16/9/2)*2)/2',scale=720:1280,setsar=1`

5. Multi-Hop Redirect Validation Against SSRF Risks

Problem with Single-Hop URL Validation

Validating only the initial URL is insufficient if that server responds with an HTTP 301/302/307/308 redirect to loopback (`127.0.0.1`), metadata services (`169.254.169.254`), or RFC1918 private IPs, which FFmpeg or fetch would otherwise follow automatically.

Refined Design

- **Hop-by-Hop Redirect Resolver (`resolveSafeExternalMediaUrl(initialUrl, maxRedirects = 5)`):**
  1. For each hop `0 .. maxRedirects`:
     - Enforce `protocol === "http:" || protocol === "https:"`.
     - Resolve all IPv4 and IPv6 records using `dns.promises.lookup(hostname, { all: true })`.
     - Reject immediately (`SSRF_BLOCKED_HOST`) if any resolved IP belongs to loopback (`127.0.0.0/8`, `::1`), link-local / cloud metadata (`169.254.0.0/16`, `fe80::/10`), private RFC1918 networks (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `fc00::/7`), or carrier-grade NAT.
     - Issue a `HEAD` or `GET` with `Range: bytes=0-1` and `redirect: "manual"`.

    - If the response is a 3xx redirect: read `Location`, resolve relative URLs, and loop to validate the next hop.
2. Pass only the final verified terminal URL to `ffprobe` and `ffmpeg` with `-protocol_whitelist file,http,https,tcp,tls,crypto`.

6. Reference-Counted Output Cleanup (Never Interrupt Active Preview or Download Sessions)

Problem with Fixed-Timer Deletion

A background interval deleting output directories at `createdAt + 30 minutes` breaks in-flight video playback and active file downloads.

Refined Design

- **Lease & Reference Tracking on `OpenShortsJobRecord`:**
    - `inputStagingDir`: Deleted immediately once rendering completes or fails.
    - `outputArtifactDir`: Retained while job is active or being previewed/downloaded.
    - `lastAccessedAt`: Updated on every `/api/status/:jobId` and `/api/artifacts/:jobId/:clipFile` request.
    - `activeStreamCount`: Incremented when an artifact HTTP response begins, decremented on `res.on("close" | "finish")`.
- **Safe Eviction Condition:**

  The periodic cleanup worker only deletes `outputArtifactDir` when:

  1. `job.activeStreamCount === 0` (no active streaming/download HTTP connection in flight), **and**
  2. `Date.now() - job.lastAccessedAt > JOB_IDLE_RETENTION_MS` (30 minutes of complete inactivity).

Updated Implementation Sequence

1. **Step 1 (`/src/utils/pipelineLogic.ts` & `/src/utils/pipelineLogic.test.ts`):** Update shared contracts, `mapOpenShortsClipsToArtifacts` per-clip artifact URL binding, and adaptive 9:16 filter calculations.
2. **Step 2 (`/server.ts`):** Startup binary check, raw binary positional chunk upload, multi-hop SSRF validation, adaptive 9:16 FFmpeg execution, and reference-counted output streaming.
3. **Step 3 (`/src/viewmodels/useOpenShortsViewModel.ts`, `/src/components/UploadsScreen.tsx`, `/src/components/PodcasterFrame.tsx`):** Raw binary chunk upload, comprehensive status polling, and clip preview wiring.
4. **Step 4 (`/android/app/src/main/java/com/fluxclip/studio/OpenShortsViewModel.kt` & `FluxClipScreen.kt`):** SAF pickers, duration inspection, raw binary upload streaming, and Compose UI parity.
5. **Step 5 (Verification):** Unit tests, TypeScript compilation & lint checks, and 20-scenario end-to-end verification.